

Contamination Ratio (CR) Analysis

[RStudio + Windows Edition](#)

msProteomiX — timsTOF Pro IM-MS Contamination Evaluation

This guide is designed for **Windows + RStudio** users. All operations can be completed within RStudio — no command-line knowledge required.

Table of Contents

1 Overview

2 One-Time Setup (5 min)

2.1 Install R + RStudio

2.2 Install msProteomiX

2.3 Install Python + alphasims

2.4 Verify Everything

3 Complete Workflow (Step by Step)

3.1 Create Project

3.2 Extract CR Data from .d Files

3.3 Interactive Grouping & Visualization

3.4 View Results

4 Understanding the Results

4.1 IM-m/z Density Maps

4.2 CR & ALIF Gradient Profiles

4.3 Cross-Sample Comparison

4.4 Metric Robustness

5 Troubleshooting

6 Parameter Reference

Appendix: CR Formula

1 Overview

What is CR?

CR (Contamination Ratio) evaluates sample cleanliness by measuring **singly-charged contaminants** vs **multi-charged peptide signals** in the IM-m/z space of timsTOF Pro data.

Key Insight: In trapped ion mobility, singly-charged contaminants (polymers, detergents, noise) sit **above** the peptide corridor in the IM-m/z plane. A dividing line separates these two populations.

$$CR = \Sigma I_{above} / \Sigma I_{below} \times 100\% \text{ — lower is cleaner.}$$

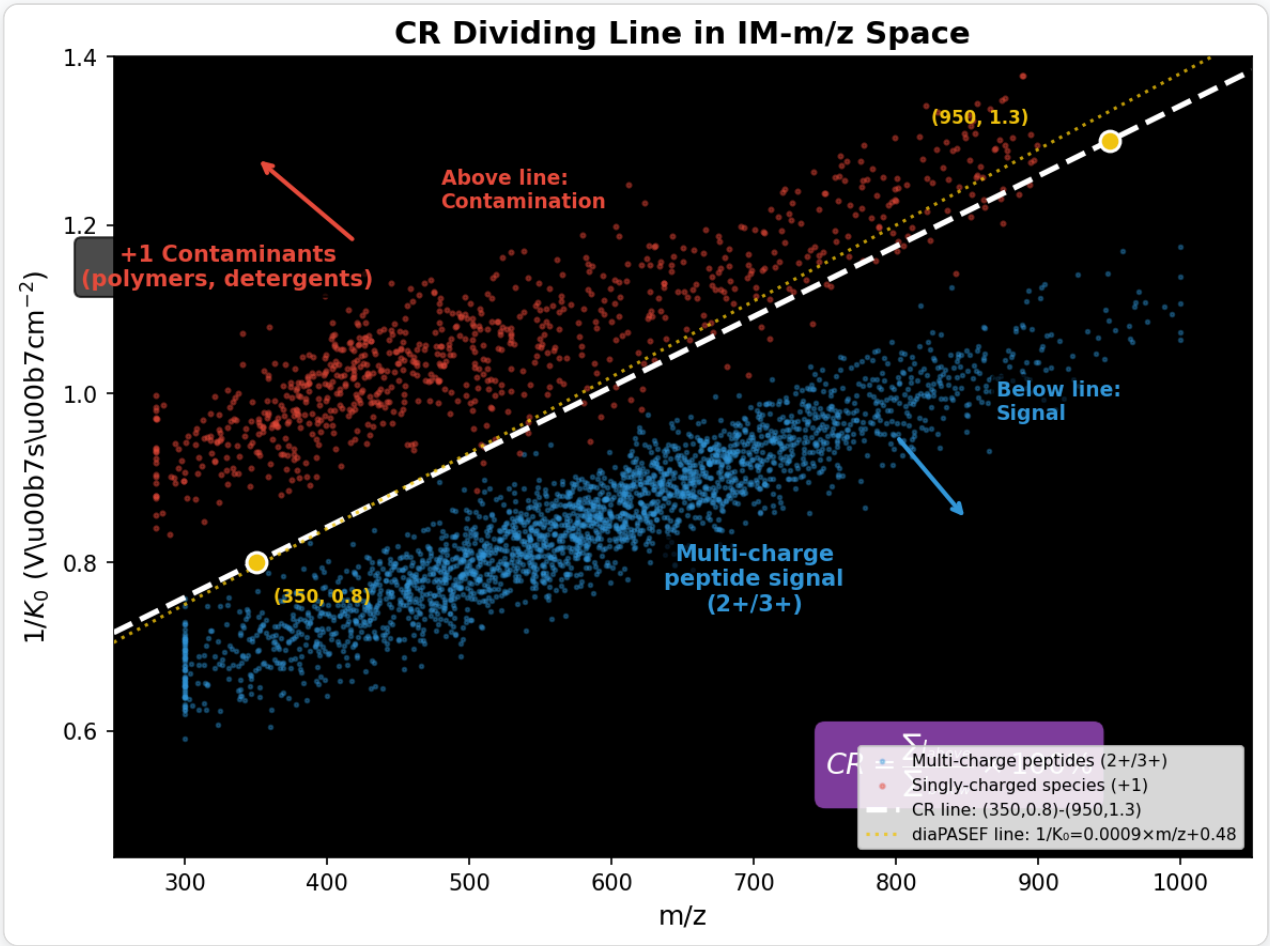


Fig. 1: CR dividing line in IM-m/z space. Above: singly-charged contaminants. Below: multi-charge peptide signal.

Workflow Overview

The entire workflow runs in **RStudio**:

Step	What Happens	Time
1	Create project & copy .d files	1 min

2	<code>extract_cr_data()</code> extracts precursors & computes CR	2-5 min/file
3	Run <code>33_contamination_ratio.R</code> — interactive grouping + plots	1 min
4	View HTML report in <code>output/</code>	—

2 One-Time Setup (5 min)

You only need to do this once on your Windows computer.

2.1 Install R + RStudio

1 Download and Install

- **R** (4.3+): <https://cran.r-project.org> — click "Download R x.x.x for Windows"
- **RStudio**: <https://posit.co/download/rstudio-desktop/> — download the Windows installer

Install both with default settings.

2.2 Install msProteomiX

2 Open RStudio, paste in Console

```
# Install msProteomiX (one-liner)
if (!requireNamespace("remotes")) install.packages("remotes")
remotes::install_github("BayOmics/msProteomiX")

# Install reticulate for Python integration
install.packages("reticulate")
```

```
# Expected output:
* DONE (msProteomiX)
=== msProteomiX v0.3.0 ===
```

2.3 Install Python + alphasims

3 Option A: Install from RStudio (easiest)

```
# In R console:
reticulate::install_miniconda() # Install Miniconda automatically
reticulate::py_install("alphasims") # Install alphasims
```

Tip: This installs a private Python environment managed by R. You don't need to install Anaconda or know any Python commands.

Option B: Use existing Anaconda (if you already have it)

If you already have Anaconda/Miniconda installed, just install alphasims:

```
# In Anaconda Prompt (not R):
pip install alphasims
```

RStudio's `reticulate` will auto-detect your Anaconda installation.

2.4 Verify Everything

4 Run in R Console

```
library(msProteomiX) # Should print version banner
library(reticulate)
py_config() # Should show Python path
py_module_available("alphasims") # Should return TRUE
```

```
=== msProteomiX v0.3.0 ===
Multi-Engine Mass Spectrometry Proteomics Data Processing
[1] TRUE
```

```
If py_module_available("alphatims") returns FALSE , run  
reticulate::py_install("alphatims") and restart RStudio.
```

3 Complete Workflow (Step by Step)

3.1 Create Project

1 Create a project folder

```
library(msProteomiX)  
create_project("D:/Projects/CR_Analysis")  
# This creates:  
#   D:/Projects/CR_Analysis/  
#   ├── scripts/      (user scripts including 33_contamination_ratio.R)  
#   ├── wkdir/        (put your data here)  
#   └── output/       (results go here)
```

2 Copy your .d files

Copy your timsTOF `.d` folders into `wkdir/` or keep them in their original location (you'll specify the path later).

Example directory structure:

```
D:/data/  
├─ HeLa_10ng_Rep1.d/  
├─ HeLa_10ng_Rep2.d/  
├─ MouseBrain_DDM_Rep1.d/  
├─ MouseBrain_DDM_Rep2.d/  
├─ MouseBrain_FFHE_Rep1.d/  
└─ MouseBrain_FFHE_Rep2.d/
```

3.2 Extract CR Data from .d Files

3 Run extraction in R Console

```
library(msProteomiX)

# Process all .d files in a directory (auto-discovers files)
extract_cr_data(
  input_path  = "D:/data/",
  output_dir  = "wkdir/cr_output",
  n_slices    = 10,          # 10 RT slices per file
  export_slices = TRUE       # Export CSVs for density maps
)
```

```
>>> Running extract_precursors.py ...
Input:  D:/data/
Output: D:/Projects/CR_Analysis/wkdir/cr_output
Slices: 10
Python: 3.10.12
alpatims: 1.1.1
[INFO] Processing: Hela_10ng_Rep1.d
[INFO]   Loaded 2325916050 detector events
[INFO]   Slice 1/10: CR=6.4%, TIC(log2)=25.5
...
>>> Done! 6 summary file(s) in: wkdir/cr_output
```

RStudio Tip: This step may take 2-5 minutes per .d file (depending on file size). You'll see progress messages in the Console. Don't close RStudio while it's running.

3.3 Interactive Grouping & Visualization

4 Open and Source the script

In RStudio, open `scripts/33_contamination_ratio.R`, then click the **Source** button (top-right of the editor).

The script will:

1. **Auto-discover** all CR result files in `wkdir/cr_output/`

2. List your samples in the Console:

```
=== Step 1: Auto-discovering CR results ===
>>> Found 6 sample(s):
      [1] HeLa_10ng_Rep1
      [2] HeLa_10ng_Rep2
      [3] MouseBrain_DDM_Rep1
      [4] MouseBrain_DDM_Rep2
      [5] MouseBrain_FFHE_Rep1
      [6] MouseBrain_FFHE_Rep2
```

3. Interactive grouping wizard — you assign groups at the prompt:

```
=== Step 2: Sample grouping ===
=====
Samples to group:
-----
      [1] HeLa_10ng_Rep1      [2] HeLa_10ng_Rep2
      [3] MouseBrain_DDM_Rep1  [4] MouseBrain_DDM_Rep2
      [5] MouseBrain_FFHE_Rep1 [6] MouseBrain_FFHE_Rep2
=====
>>> Enter sample numbers (e.g. 1-2,5): 1-2
>>> Enter group name: HeLa
>>> Enter sample numbers (e.g. 1-2,5): 3-4
>>> Enter group name: DDM
>>> Enter sample numbers (e.g. 1-2,5): 5-6
>>> Enter group name: FFHE
>>> Grouping complete? (y/n): y
```

4. Auto-generate all plots and HTML report

Flexible: You can define **any number of groups** with any names. The grouping is saved to `cr_group_info.csv` and reused on subsequent runs.

3.4 View Results

5 Check the output/ folder

```
output/
├─ CR_Analysis_CR_Gradient.png    ← CR vs RT curve
├─ CR_Analysis_CR_TIC.png        ← TIC intensity
```

└ CR_Analysis_CR_Heatmap_*.png ← IM-m/z density maps
└ CR_Analysis_CR_Report.html ← Combined HTML report

Double-click the *_CR_Report.html file to view in your browser.

4 Understanding the Results

4.1 IM-m/z Density Maps

Each density map shows ion distribution in the IM-m/z plane for one RT slice. The **white dividing line** separates contaminants (above) from peptide signal (below). CR is annotated on each panel.

4.1.1 HeLa 10ng (clean reference)

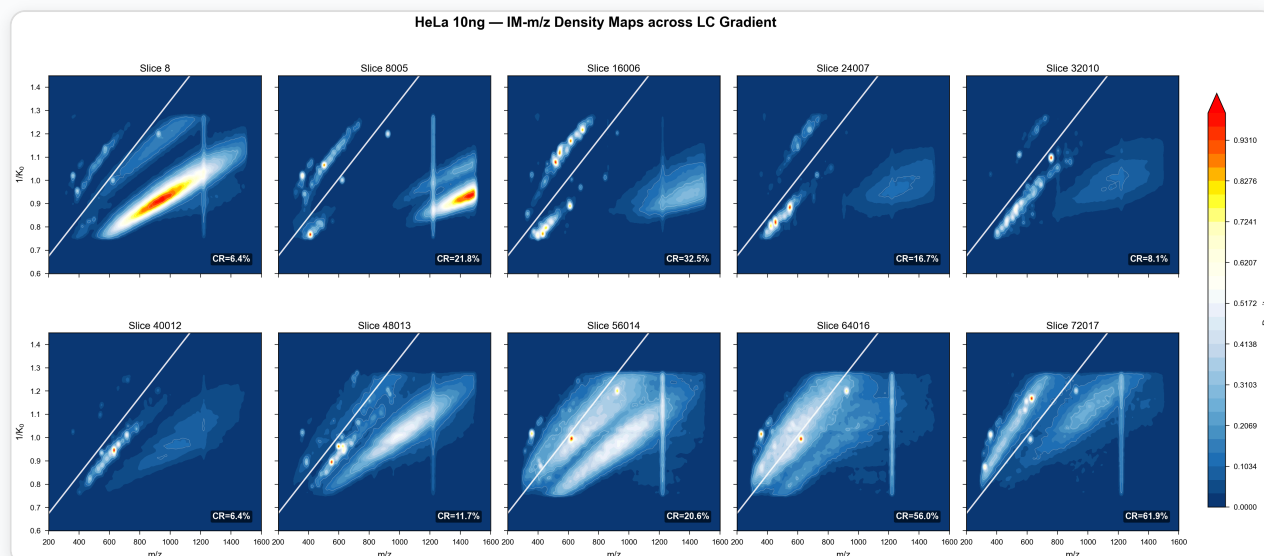


Fig. 2a: HeLa 10ng IM-m/z density maps across LC gradient. CR ranges from 6.4% (mid-gradient, cleanest) to 61.9% (late gradient).

4.1.2 Mouse Brain DDM2 (high contamination)

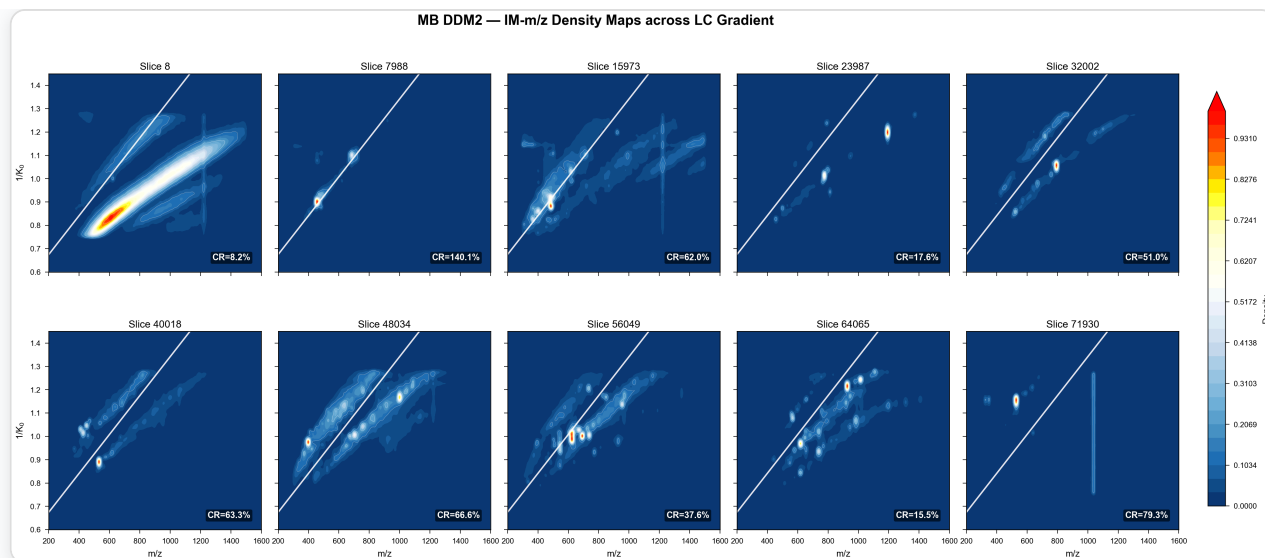


Fig. 2b: Mouse Brain DDM2 density maps. CR ranges from 8.2% to 140.1%. Prominent contaminant bands above the dividing line.

4.1.3 Mouse Brain FFHE2 (FFPE tissue)

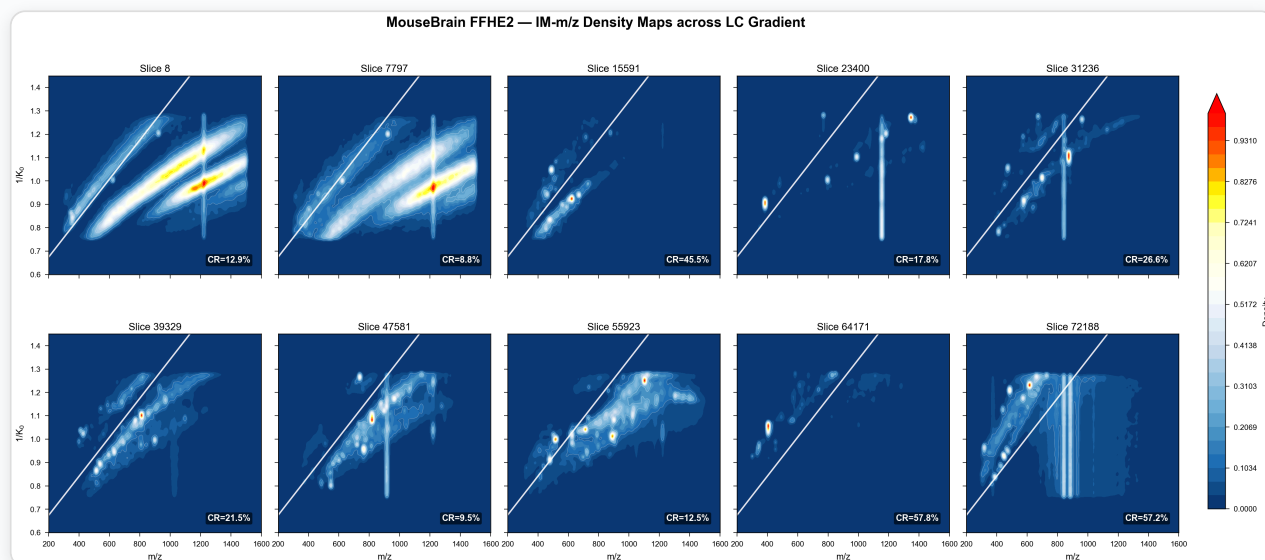


Fig. 2c: Mouse Brain FFHE2 density maps. Moderate contamination with tail-gradient enrichment.

4.2 CR & ALIF Gradient Profiles

These plots show how contamination changes across the LC gradient for each sample type.

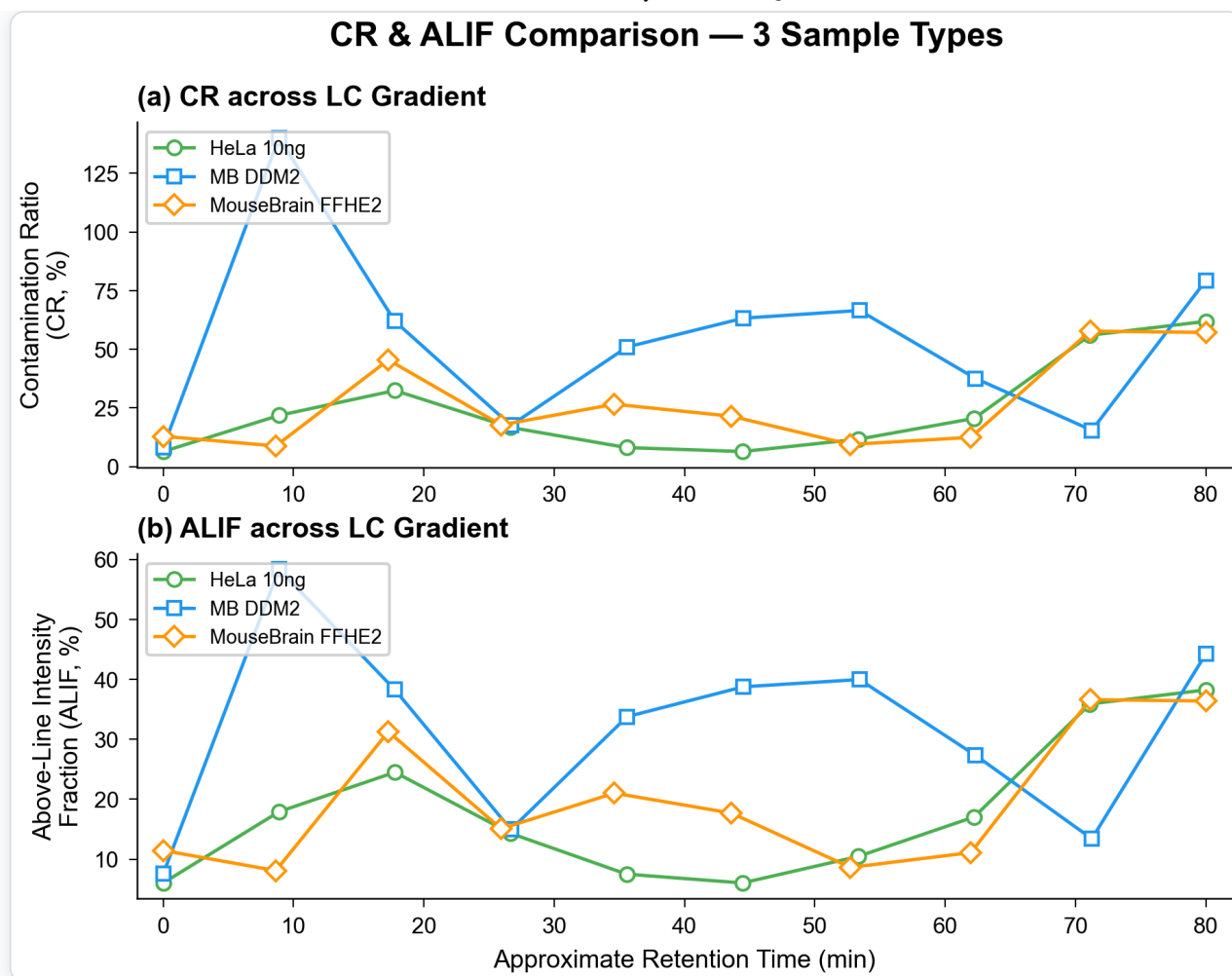


Fig. 3: CR (upper) and ALIF (lower) profiles across LC gradient. MB DDM2 (blue) shows elevated contamination throughout.

Interpreting the profiles:

- **U-shaped curve** (HeLa): Normal — gradient start/end have less peptide signal
- **Elevated baseline** (DDM2): Systematic contamination — sample prep issue
- **CR > 100%**: Contaminant signal exceeds peptide signal (severely dirty)

4.3 Cross-Sample Comparison

3×3 Heatmap Matrix

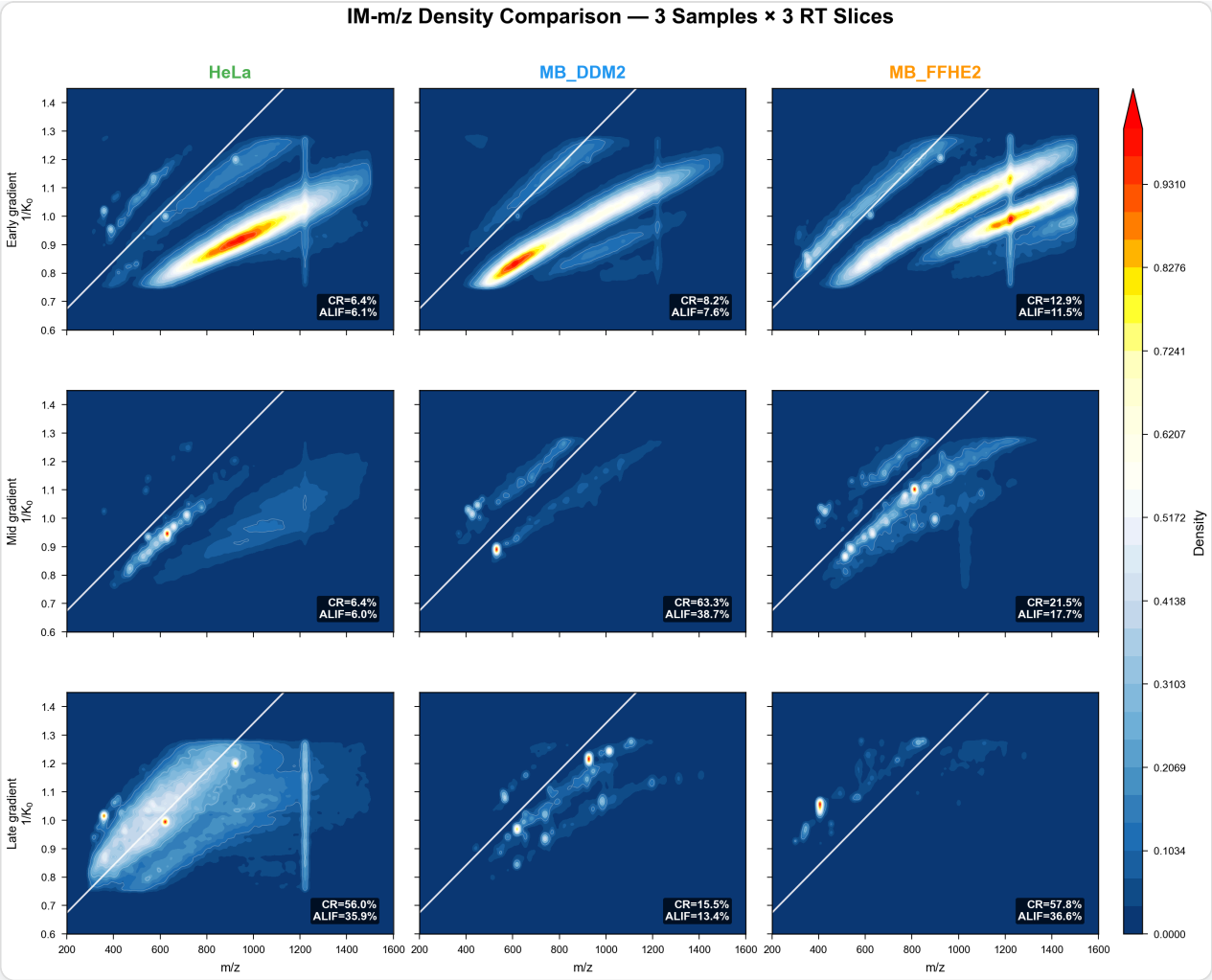


Fig. 4: 3 samples x 3 RT slices (early, mid, late). Same color scale. MB DDM2 shows consistently higher contamination.

Summary Statistics

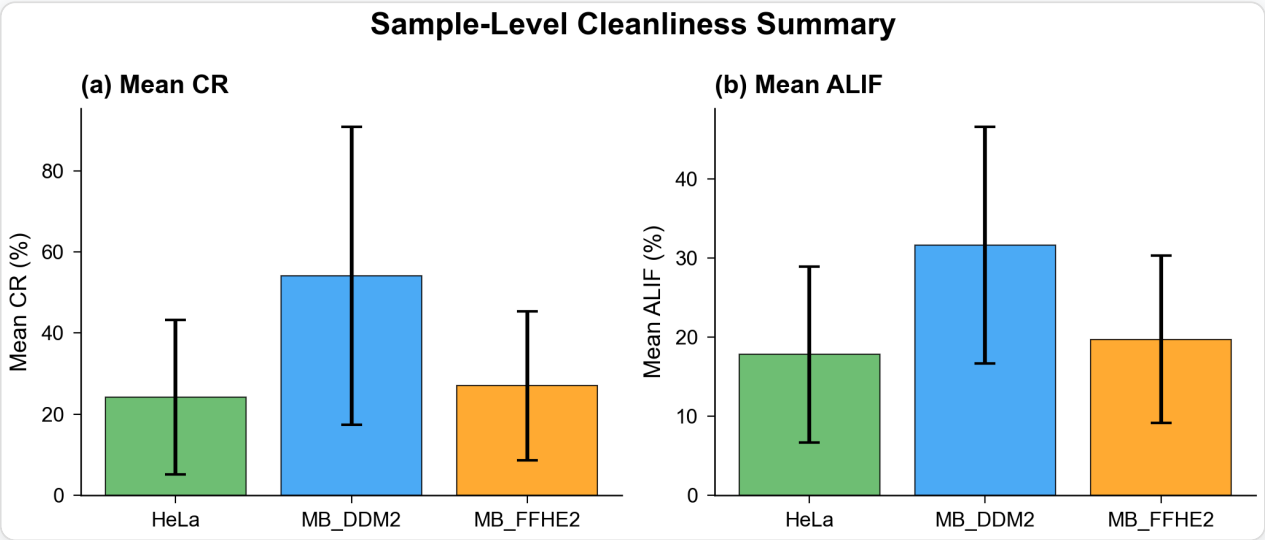


Fig. 5: Mean CR and ALIF with SD error bars.

4.4 Metric Robustness

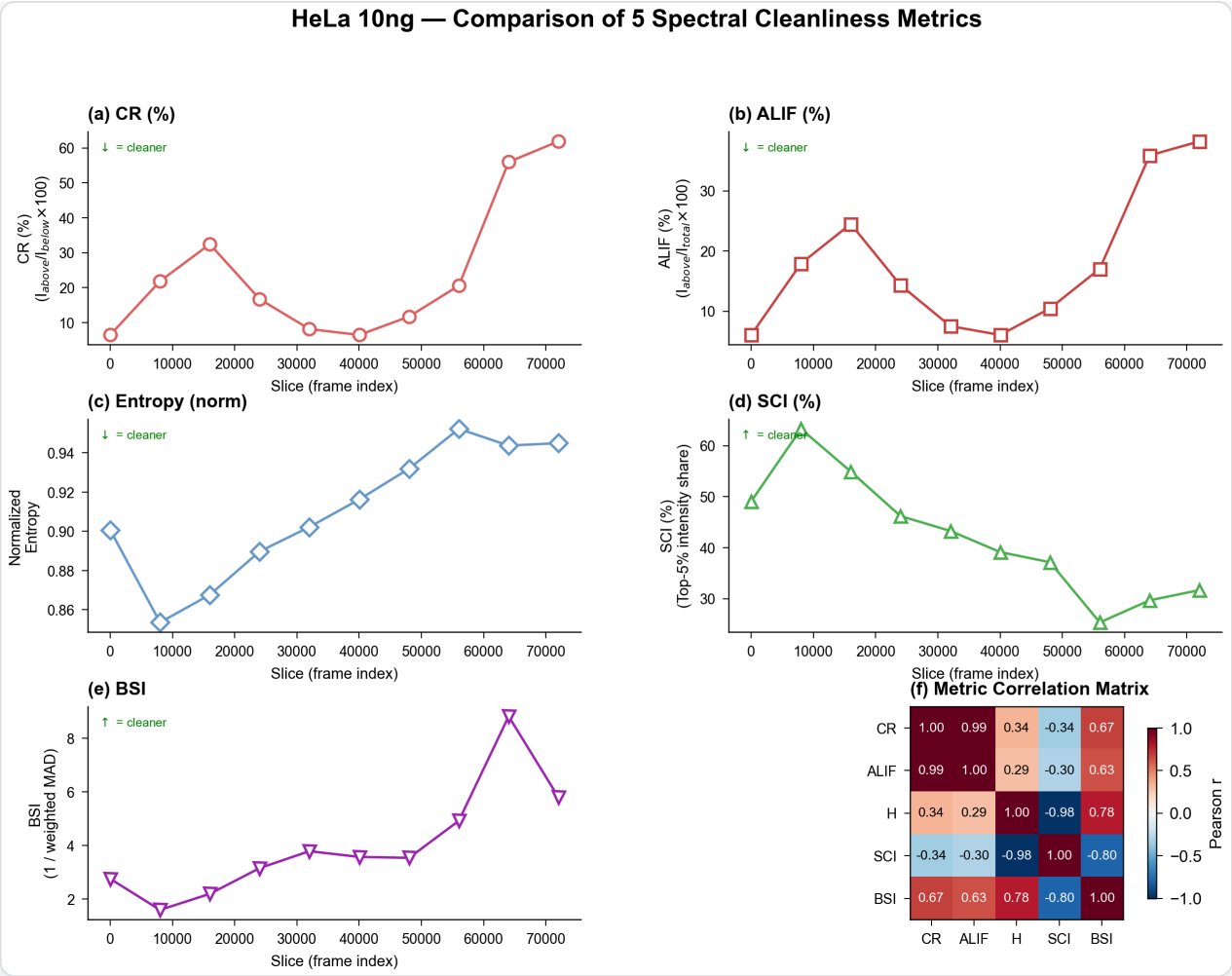


Fig. 6: Comparison of 5 spectral cleanliness metrics. CR and ALIF are highly correlated ($r=0.99$) and outperform alternatives.

Conclusion: CR is the best single indicator with 10× dynamic range. ALIF (bounded [0, 100%]) is recommended as a complementary metric.

5 Troubleshooting

Problem	Cause	Solution
<code>reticulate::py_config()</code> shows no Python	No Python installed	Run <code>reticulate::install_miniconda()</code> in R
<code>py_module_available("alphatims")</code> is FALSE	alphatims not installed	Run <code>reticulate::py_install("alphatims")</code>

<code>extract_cr_data()</code> hangs	Large .d file (>10GB)	Normal — wait 5-10 min. Check Console for progress.
CR values all very high (>50%)	Possible contamination or wrong dividing line	Check sample prep; try adjusting <code>LINE_POINT1/2</code>
No heatmap slice CSVs found	<code>export_slices = FALSE</code> in extract step	Re-run <code>extract_cr_data()</code> with <code>export_slices = TRUE</code>
Error: CR output directory not found	Path mismatch	Check <code>CR_DIR</code> in script matches <code>output_dir</code> in extract
RStudio says "function not found"	Package not loaded	Run <code>library(msProteomiX)</code> first

6 Parameter Reference

`extract_cr_data()`

Parameter	Default	Description
<code>input_path</code>	(required)	Path to .d folder, .hdf file, or a directory containing them
<code>output_dir</code>	"wkdir/cr_output"	Where to save results
<code>n_slices</code>	10	Number of equally-spaced RT slices
<code>export_slices</code>	TRUE	Export per-slice CSV files for density maps
<code>heatmap_rts</code>	NULL	Specific RT values (min) for heatmap. NULL = all slices.

line_points	"350,0.8,950,1.3"	CR dividing line: mz1,im1,mz2,im2
python	NULL	Python path (auto-detected via reticulate)

Script 33_contamination_ratio.R

Variable	Default	Description
PROJECT_NAME	"CR_Analysis"	Prefix for output filenames
CR_DIR	"wkdir/cr_output"	Directory with *_cr_summary.csv files
OUTPUT	"output"	Output directory for plots and report
LINE_POINT1	c(350, 0.8)	Dividing line point 1: (m/z, 1/K ₀)
LINE_POINT2	c(950, 1.3)	Dividing line point 2: (m/z, 1/K ₀)

Appendix: CR Formula

Physical Basis

In TIMS, $1/K_0 \propto \text{CCS} / z$. At the same m/z, singly-charged ions ($z=1$) have ~26% higher $1/K_0$ than doubly-charged ions ($z=2$).

Dividing Line

$$(350, 0.8) \rightarrow (950, 1.3)$$

$$1/K_0 = 0.000833 \times m/z + 0.5083$$

CR Computation

- $1/K_0 \geq \text{line value} \rightarrow \text{Contaminant}$
- $1/K_0 < \text{line value} \rightarrow \text{Signal}$

$$\text{CR} = \Sigma I_{\text{contam}} / \Sigma I_{\text{signal}} \times 100\%$$

msProteomiX v0.3.0 — BayOmics
RStudio + Windows Edition
Generated by generate_cr_manual_rstudio.py